

Testbericht – Smart Restaurant / Orbit

Stand: 06.10.2026

1. Überblick

Testprojekt	SmartRestaurant.Tests (xUnit, .NET 10)
Testobjekt	SmartRestaurant.Api – REST-Controller
Datenbank im Test	EF Core In-Memory -Provider (frische Datenbank pro Test, kein MySQL nötig)
Automatisierung	GitHub-Actions-Workflow <code>.github/workflows/tests.yml</code> , läuft bei jedem <code>git push</code> und Pull Request
Ergebnis	10 Tests (11 Fälle) – 11 bestanden, 0 fehlgeschlagen

Lokal ausführen:

```
dotnet test SmartRestaurant.Tests
```

2. Testfälle

Jeder Test folgt dem Muster *Arrange – Act – Assert*. `TestDb.Create()` legt Rollen, Tisch-/Bestellstatus, eine Kategorie mit zwei Artikeln (Cola 3,50 €, Bier 4,20 €), zwei Tische und zwei Mitarbeiter (einer aktiv, einer deaktiviert) an.

#	Test	Bereich	Was wird geprüft	Ergebnis
1	Login_MitRichtigemPasswort_LiefertMitarbeiterUndRolle	Login	Richtiges BCrypt-Passwort → 200 OK mit Mitarbeiter-ID und Rolle	✓
2	Login_MitFalschemPasswort_IstUnauthorized	Login	Falsches Passwort → 401	✓
3	Login_DeaktivierterMitarbeiter_IstUnauthorized	Login	Deaktivierter Mitarbeiter kann sich auch mit richtigem Passwort nicht anmelden	✓
4	Create_OhnePositionen_IstBadRequest	Bestellungen	Bestellung ohne Positionen → 400, nichts wird gespeichert	✓
5	Create_SpeichertBestellungMitPreisAusDbUndBesetztTisch	Bestellungen	Bestellung wird mit Status „Offen“ angelegt, Preise kommen aus der DB ($2 \times 3,50 + 4,20 = 11,20$ €), Tisch wird „besetzt“	✓
6	SetStatus_Bezahlt_GibtTischFrei	Bestellungen / Bezahlung	Status „Bezahlte“ speichert den Abschlusszeitpunkt und gibt den Tisch frei	✓
7	ArtikelCreate_MitNegativemPreis_IstBadRequest	Admin – Artikel	Negativer Preis wird abgelehnt	✓
8	ArtikelCreate_GleicherNameInKategorie_IstConflict	Admin – Artikel	Doppelter Name in derselben Kategorie (ohne Groß-/Kleinschreibung, getrimmt) → 409	✓
9	TischCreate_MitUnguealtigerKapazitaet_IstBadRequest (1 und 9)	Admin – Tische	Kapazität außerhalb von 2–8 wird abgelehnt (Grenzwerte)	✓✓

#	Test	Bereich	Was wird geprüft	Ergebnis
10	GetWoche_ZaehltNurBezahlteBestellungenDerLetzten7Tage	Statistik	Wochenumsatz und Bestseller berücksichtigen nur bezahlte Bestellungen der letzten 7 Tage	✓

Einschränkung: Der In-Memory-Provider prüft keine MySQL-Regeln wie Fremdschlüssel oder Unique-Indizes. Die Tests prüfen daher die Programmlogik, nicht die Datenbankregeln – dafür wären Integrationstests der nächste Schritt.

3. Automatisierte Testmethoden für dieses Projekt

Methode	Eingesetzt?	Zweck in diesem Projekt
Unit-Tests	✓ (diese Testsuite)	Controller-Logik isoliert testen: Validierung, Geschäftsregeln (Tischstatus, Preise, Statistik). Schnell, keine Infrastruktur nötig.
Integrationstests	möglich	API mit <code>WebApplicationFactory</code> gegen einen echten MySQL-Container (z. B. Testcontainers) starten, um Routing, JSON, SQL und DB-Constraints zu prüfen, die der In-Memory-Provider ignoriert.
Komponententests (Frontend)	möglich	Blazor-Komponenten (<code>Order.razor</code> , <code>Bar.razor</code>) mit bUnit testen – z. B. Warenkorbsumme, +/- Buttons.
End-to-End-/UI-Tests	möglich	Playwright steuert einen echten Browser: Anmelden → Bestellen → Bezahlen. Am langsamsten, prüft aber das Gesamtsystem.
Continuous Integration	✓	GitHub Actions baut und testet bei jedem Push; ein fehlgeschlagener Test markiert den Commit rot.

4. Nutzen für Qualitätssicherung und Entwicklungsprozess

- **Frühe Fehlererkennung** – Fehler (z. B. ein Tisch, der nach dem Bezahlen nicht freigegeben wird) fallen Minuten nach dem Push auf, nicht erst bei der Präsentation.
- **Sicheres Refactoring / Schutz vor Regressionen** – die Controller können umgebaut werden; die Tests zeigen sofort, ob sich das Verhalten geändert hat.
- **Lebende Dokumentation** – die Testnamen beschreiben die Geschäftsregeln („Kapazität muss 2–8 sein“, „nur bezahlte Bestellungen zählen“).
- **Zusammenarbeit im Team** – bei mehreren Branches (`Frontend-bar-Service` , `Tests` , ...) stellt CI sicher, dass niemand Code mergt, der bestehende Funktionen kaputt macht.
- **Reproduzierbarkeit** – durch die In-Memory-Datenbank sind die Tests unabhängig von einer lokalen MySQL-Installation; jeder Lauf startet im gleichen Zustand.

5. Testgetriebene Entwicklung (TDD)

TDD folgt einem kurzen Zyklus: **Red** (einen fehlschlagenden Test für die nächste Anforderung schreiben) → **Green** (nur so viel Code schreiben, dass der Test besteht) → **Refactor** (aufräumen, während die Tests grün bleiben).

Argumente für TDD

- Anforderungen werden *vor* dem Programmieren geklärt – z. B. wird „Was passiert bei einer leeren Bestellung?“ zu Test Nr. 4.
- Führt zu testbarem, lose gekoppeltem Code (Controller hängen von einem injizierbaren `AppDbContext` ab).
- Eine hohe Testabdeckung entsteht automatisch, statt am Ende nachgeholt zu werden.
- Weniger Fehler und weniger Debugging-Aufwand in späteren Phasen.

Argumente dagegen / Grenzen

- Höherer Anfangsaufwand; am Anfang eines kurzen Schulprojekts langsamer.
- Weniger sinnvoll für UI-Layout und exploratives Prototyping.
- Tests müssen gepflegt werden, wenn sich Anforderungen ändern.

Fazit: Für die Geschäftslogik der API (Bestellungen, Bezahlung, Validierung, Statistik) eignet sich TDD gut und zahlt sich schnell aus. Für die Blazor-Oberfläche ist es pragmatischer, Tests nach der Implementierung zu schreiben (oder E2E-Tests für die Hauptabläufe).